

Redrob Candidate Intelligence Ranker

Team Name: Team Redrob Ranker | Problem: AI/Data candidate ranking

One-line idea

An offline, explainable learning-to-rank system that finds the best 100 ML/search/recommendation candidates from 100,000 profiles.

No live LLM

Ranking and reasoning run offline with deterministic Python code.

Evidence-first

Career proof beats raw keyword matching.

Fake-risk aware

Suspicious timelines and weak profiles are penalized.

Solution Overview

Proposed solution and key differentiator

- Build a candidate intelligence ranker, not a simple keyword matcher.
- Extract structured evidence from title, career history, skills, platform signals, logistics, and risk signals.
- Train a lightweight pairwise reranker on reviewed labels from 520 examples.
- Generate deterministic explanations from actual candidate evidence, not LLM hallucinations.
- Validate the final top 100 with fake-risk, experience, title, reasoning, and official CSV checks.

JD Understanding & Candidate Evaluation

How fit is evaluated beyond keyword matching

Must-have fit

ML/search/recommendation background, 5-9 years, production ownership.

High-value proof

Ranking/retrieval systems, NDCG/MRR, A/B tests, offline-online evaluation.

Risk checks

Mismatched experience, keyword stuffing, stale profiles, suspicious timelines.

- Strong candidates prove they have owned real relevance systems.
- Medium candidates have adjacent ML/RAG/semantic-search experience but limited ranking ownership.
- Weak candidates may have ML keywords but no production search/recs evidence.

Ranking Methodology

How candidates are retrieved, scored, and ranked

1. Feature extraction

Turn each profile into fit, skill, evidence, logistics, and risk features.

2. Reviewed labels

Use 520 reviewed labels to teach what good ranking looks like.

3. Pairwise reranking

Learn candidate A should rank above candidate B when label A is better.

4. Final sort

Blend learned score with rule score and produce top 100.

- Algorithm: standardized linear reranker trained with pairwise logistic loss.
- Final score blends learned signal with the problem-specific feature score.
- Runs fully offline on CPU.

Explainability & Data Validation

How decisions are explained and hallucinations are avoided

- Reasoning is template-based and generated from actual profile fields.
- Evidence terms are selected from detected career hits like semantic search, learning-to-rank, offline metrics, and recommendation systems.
- No LLM is called during ranking or explanation generation.
- Fake-risk checks catch mismatched years, suspicious timelines, domain mismatch, and weak keyword-only profiles.
- Final QA: fake-risk > 0.10 in top 100 = 0; under-5-years = 0.

End-to-End Workflow

From JD input to ranked candidate output

1

Read JD and challenge docs

2

Stream candidate JSONL

3

Extract profile, career, skill, signal, and risk features

4

Train reranker on reviewed labels

5

Rank 100,000 candidates

6

Generate explanations

7

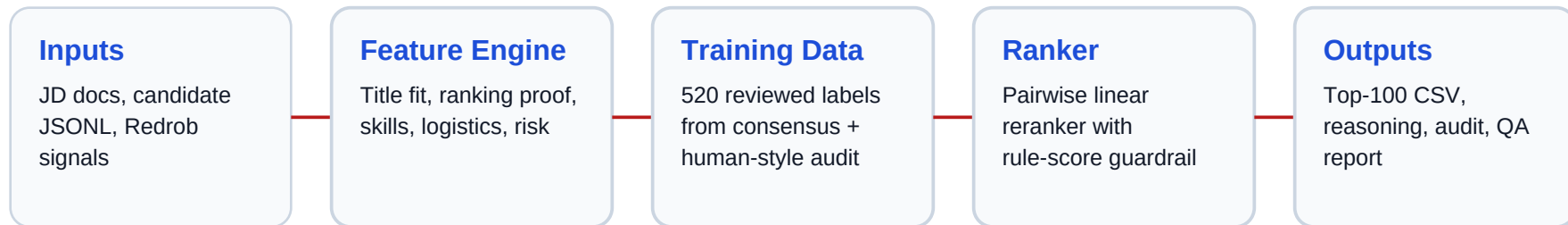
Run validator and QA

8

Package submission assets

System Architecture

Offline, reproducible, explainable



- No external inference service is needed.
- Every output can be traced back to features, labels, and candidate evidence.
- The audit file makes top-ranked decisions inspectable.

Results & Performance

Ranking quality and constraints

Validator

Passed

Rows

100

Fake-risk leaks

0

Under-5 leaks

0

Pair accuracy

0.961

MAP label ≥ 3

0.976

Technologies Used

Small, explainable stack

- Python standard library for streaming JSONL, scoring, CSV writing, and offline ranking.
- Custom feature extraction for ranking/retrieval/evaluation/production evidence.
- Pairwise linear reranker implemented without heavy ML dependencies.
- ReportLab for generated PDFs and documentation assets.
- pdfplumber/pypdf for PDF extraction and QA.
- No live LLM, GPU, cloud database, or paid API needed at submission time.

Submission Assets

What is included in the final package

- final/team_redrob_ranker_v2_final.csv - final upload CSV.
- final/audit_top300_v2_final.csv - evidence and feature audit.
- final/qa_report_final.json - validator and quality checks.
- all_csv/ - labels, predictions, audits, and variant outputs.
- all_json/ - models, metrics, and reports.
- code/ - complete scripts used to train, rank, validate, and document.
- docs/ - approach PDF, PPT-content PDF, workflow notes, and implementation report.

Why This Should Stand Out

Practical, defensible, and aligned to the problem

- It solves the real ranking problem, not just resume keyword search.
- It balances ML learning with hard domain rules and fake-profile checks.
- It creates explanations from evidence, so judges can trust the ranking.
- It is lightweight and reproducible under hackathon constraints.
- It ships a complete audit trail: labels, model, metrics, code, final CSV, and QA proof.

Final message: evidence-first candidate intelligence, not generic matching.